

Document number: P2570R0
Date: 2022-06-08
Audience: SG21
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

On side effects in contract annotations

This paper presents a range of approaches to dealing with side effects in contract predicates proposed in [\[P2521R2\]](#).

1. Overview

{ovr}

Because the choice of syntax for contract annotations has no consensus yet, in this paper we use a placeholder notation:

```
int select(int i, int j)
    PRE(is_even(i))           // precondition
    PRE(is_even(j))
    POST(r: is_even(r))      // postcondition; r names the return value
{
    ASSERT(is_nonnegative(_state)); // assertion; not necessarily an expression: may be a statement

    if (_state == 0) return i;
    else             return j;
}
```

Ideally, a contract annotation reads something like the above, and we intuitively assume that functions `is_even()` and `is_nonnegative()` have no side effects, which means that unless we read their return value (and take a branch based on it) the programmer will not be able to tell if this function has been invoked and how many times. The situation is different when the programmer puts a side effect into the predicate and starts relying on it:

```
void f()
    PRE(++global_call_counter < 100);
```

There is no easy way to prevent the programmer from putting such side effects. Note that side effects may be hidden behind the function call:

```
void f()
    PRE(less_than_100_calls_so_far());
```

Therefore, we need to specify what happens when a contract predicate contains a side effect.

The range of possible choices includes:

- Make the program ill-formed if the predicate cannot be proven to be side-effect free.
- Use the C-assert model: just evaluate the expressions (with side effects) in one mode, and not evaluate them in the other.
- In *Check_and_abort* mode, allow the predicates to be evaluated zero, one, or more times — at the implementation's discretion — which means that the side effects become unreliable.

1.2. Design criteria

{ovr.cri}

We want to remain faithful to the objectives outlined in [\[P2182R0\]](#), and called the MVP: while providing a useful — albeit small — addition, remain open for the widest possible range of possible future directions. In consequence:

- We try to avoid specifying semantics for aspects that remain controversial. Where two possible future directions exist, we want to remain open for both.
- For aspects that we do not want to define, we prefer making them ill-formed to "Undefined Behavior" (UB) or "Ill Formed, no diagnostic required" (IF-NDR). This is to spare the programmers runtime surprises, and to save the syntax space for future extensions.
- We prefer defining things as UB to defining them as unspecified or implementation defined. This reserves us the possibility to provide a uniform, platform-independent semantics in the future, as well as to make such constructs ill-formed in the future.

We recognize the following three consumers of contract annotations:

1. Humans and IDEs: for informational purposes.
2. Static analysis: to report in a systematic way execution paths that lead to contract violations.
3. Runtime checks: to detect situation where a contract annotation is violated in every path actually taken at runtime, and abort the execution of the program immediately.

The approach to side effects in contract annotations should consider the effect on these three consumers.

Finally, we consider contract annotations to be a feature whose primary goal is to increase safety. This is in the context of the language that has a number of unsafe aspects. For this reason we want this feature to be safe: have as little surprising runtime behavior as possible. Also, for this feature we expect a different trade-off between safety and efficiency/expressibility than for other C++ features; therefore we are prepared to compromise the uniformity with other features, and not necessarily give in to arguments like, "treat contract predicates as any other expressions."

2. Discussion

{dis}

2.1. Classification of side effects

{dis.cls}

Our intention is that programmers would never put contract predicates whose side effects affect the logic of the program, such as:

```
void g()
  PRE(start_device_if_not_started()) // bad!
{
  use_device();
}
```

Nor contract predicates whose side effects affect the program correctness -- understood as compliance with contract annotations:

```
void f()
  PRE(less_than_100_calls_so_far()); // bad!
```

However, [\[P2388R4\]](#) lists side effects that could be considered benign:

1. Logging, which never affects subsequent computations.
2. Modifying private mutable data members for the purpose of caching function results.
3. Using mathematical functions from `<cmath>`, which store error results in global (thread-local) variable `errno`.
4. Performing scoped locking inside the function, which may affect the execution of other threads.
5. Triggering a contract violation handler when runtime-checking the precondition of the function called in the predicate.

These benign side effects do not affect the mental model for contract annotations: from the programmer's perspective the predicates are still not affecting the program logic. More, it is often impossible for programmers to even know if the function they use has any side effects. For instance, the specification of `std::vector<T>::size()` does not prevent the implementations from performing side effects, such as logging.

The C++ Standard currently has the following definition of "side effect" in [\[intro.execution\]](#):

Reading an object designated by a `volatile` glvalue ([\[basic.lval\]](#)), modifying an object, calling a library I/O function, or calling a function that does any of those operations are all *side effects*, which are changes in the state of the execution environment. *Evaluation* of an expression (or a subexpression) in general includes both value computations (including determining the identity of an object for glvalue evaluation and fetching a value previously assigned to an object for prvalue evaluation) and initiation of side effects. When a call to a library I/O function returns or an access through a volatile glvalue is evaluated the side effect is considered complete, even though some external actions implied by the call (such as the I/O itself) or by the `volatile` access may not have completed yet.

Based on the above, we could introduce the classification of side effects:

1. Any side effect, as defined by the C++ Standard.
2. Any side effect *visible outside* the predicate. For instance, creating a local automatic variable and mutating it inside the predicate does not count.
3. A side effect that does not affect the program logic.

The first one is formally defined; we could also provide a definition for the second. The third class depends on the notion of "program logic" which only the programmer is aware of. For instance, logging into a file may or may not affect the program logic, depending on whether the program later decides to read the contents of the log.

2.2. Predicate side effects in static analysis

{dis.anl}

One of the motivations for adding contract support to the language is to enable the programmers to give more input to static program analysis. With this static analyzers can (1) detect more bugs, and (2) consume fewer resources. However, this will work when the information is in the form of a predicate: something that does not have side effects affecting other parts of the program.

Given that we propose two translation modes — *No_eval* and *Eval_and_abort* — when predicates have side effects, we end up with potentially two different programs: one may be correct, and the other not. Now, static analyzer has double work to do, or it has a tough decision to make: analyze the correctness in *No_eval* or *Eval_and_abort* mode? And the result for *No_eval* mode will not necessarily be applicable in *Eval_and_abort*. But will the users understand this?

2.3. Treating side effects as errors

{dis.err}

It is possible to statically guarantee that contract predicates are side-effect free, provided that we (significantly) limit the expressions to those that can be demonstrated to be side-effect-free.

As stated above, we need to provide a more relaxed alternative to the definition of a side effect. We will call it a *side-effect-free expression*:

An expression *E* is said to be *side-effect-free* unless it involves any of the following:

- reading or modifying an object designated by a `volatile` glvalue;
- modifying an object whose lifetime started before the evaluation of *E*;
- calling a library I/O function;
- calling any other function.

With this definition we can require that predicates in contract annotations must be side-effect-free expressions:

```

struct X { int j; };

void h(std::vector<int> const& vec, int i)
    PRE(i != 0) // ok
    PRE(!vec.empty()) // error: calls a function
    PRE(--i != 0) // error: modifies i whose lifetime started before the expression
    PRE(--(int&)X[i].j != 0) // ok: modifies a temporary X whose lifetime started within the expression
;

```

As we can see this is very restrictive. We cannot even express a precondition such as `!vec.empty()`. What we get in return is a guarantee that a contract predicate will never have a side effect. The restriction is too harsh for a long run, but this would only be a restriction for the MVP. It can be later relaxed in at least two ways.

Adapting this solution makes the question whether predicates should be evaluated zero or more times moot. Static analysis does not have to consider the situations (control paths) when the predicates have side effects affecting the program.

2.3.2. Interaction with `constexpr`-functions

{dis.err.con}

One could ask a question whether the definition of a side-effect-free expression could be relaxed to allow `constexpr`-functions. This however would not work, because `constexpr`-functions do not guarantee that they have no side effects for all input values:

```

constexpr int f(int i, int j)
{
    if (i == 3) // the only constexpr branch
        return j;

    std::cout << "side effect \n";
    return -1;
}

constexpr int c = f(3, 2); // ok
int i = f(0, 0); // side effect

```

2.3.3. Potential future directions

{dis.err.fut}

One way this can be extended in the future is to relax the definition of side-effect-free. Introduce a new function specifier, say `noeffect`, which puts a constraint on the function definition: all the expressions inside shall be side-effect-free. The definition of side-effect-free could be changed to:

An expression *E* is said to be *side-effect-free* unless it involves any of the following:

- reading or modifying an object designated by a `volatile` glvalue;
- modifying an object whose lifetime started before the evaluation of *E*;
- calling a library I/O function;
- calling a function other than `noeffect`-function.

This enables the users to express contract predicates like `!vec.empty()`, provided that function `empty()` is declared `noeffect`. The cost of this is that we put another fracture in the family of expressions, and require even more specifiers to be put on function declarations. This approach is similar to the one adapted for Transactional Memory support in [N4302]. It should be noted that even with this extension some intuitive uses of contract annotations will not work. For instance, when functions are called through function wrappers:

```

int algo(function<int(int)> op, int i)
    PRE(op(i) > 0); // error: function::operator() is not noeffect

```

A function wrapper like `std::function` must be prepared to handle `noeffect` and non-`noeffect` functions, assigned at runtime. Therefore, the wrapper itself cannot be `noeffect`.

Another possible future direction is to simply allow side effects in contract predicates. This door remains open if we take the conservative approach for the MVP.

2.4. Treating side effects as undefined behavior

{dis.ube}

Another approach to the problem of side effects in contract predicates is to impose no syntactic constraints on the predicates, but instead call it undefined behavior when the evaluated contract condition has a side effect. This has an unsettling consequence of introducing a yet another UB, in the context of a safety feature such as contract support.

On the positive side, this allows the implementations to evaluate the contract predicates arbitrary number of times. More, it encourages the implementations to emit diagnostics for side effects planted in contract predicates. The variant of this path is to make the program ill-formed, NDR. This enables the potential future direction of making side effects ill formed.

Another future direction is to assign a well-defined behavior to side effects: for instance, apply the C-`assert` model.

While term "undefined behavior" may be concerning, it should be noted that here we are talking about a different situation than the one that caused controversies over [P0542R5] (the "C++20 contracts"). Aggressive code transformations based on undefined behavior caused by contract annotation violations were, and remain, a source of concern. This is because contract annotations give the exact information that the optimizers need, and optimizations

like this are of interest to users and are implemented in the compilers. The danger here stems from the interaction between UB and code optimizations. This has been described in [\[P1728R0\]](#). We can call it "contract-based optimizations".

On the other hand, regarding the UB resulting from side effects in contract predicates — we can call it "side effect elision/duplication" — there is no known way for them to be used in optimizations. So, the concern of the same nature does not apply here. The remaining concern is that with appering and disappearing side effects. And the risk is even smaller when the predicates do not contain side effects. Note that the problem with "contract-based optimizations" occurs when contracts themselves are correct and only the programmers misuse the components (e.g., call functions out of contract). On the other hand, problems with "side effect elision/duplication" arise only if contract annotations themselves are fishy.

2.5. Allow side effects

{dis.sem}

Another possibility is to allow just any expression in contract predicates. Then we have to specify the semantics in the two translation modes: *No_eval* and *Eval_and_abort*. Mode *No_eval* is simple: we guarantee that the predicates are never evaluated: this implies no side effects are effectuated. This mode offers a performance guarantee.

For the other mode — *Eval_and_abort* — we can see two options. It can express one of the two guarantees:

1. We guarantee that the program will not continue upon value `false`, but we do not guarantee that the side effects of the expression will be observable, or if the expression is called twice. We will call it *abort guarantee*.
2. We guarantee the same as C `assert()` does: that the expression is simply evaluated, single time, and its side effects are guaranteed as for any other expression in the language. We will call it *eval guarantee*.

So, our primary question is what the user expects from *Eval_and_abort* mode: program abort upon a detected bug, or conditional evaluations of expressions? For instance, should the following use case be guaranteed to work?

```
bool mode_eval_and_abort = false;

void log_mode()
  PRE(mode_eval_and_abort = true)
{
  clog << "running program in mode " << (mode_eval_and_abort ? "Eval_and_abort"sv : "No_eval"sv);
}
```

The argument in favour of using *eval guarantee* (side effects executed, exactly once) is simplicity. If there is any bug in the application — not detected by contract annotations, or caused by contract annotations — compiled in *Eval_and_abort* mode, when you need to debug the problem, it is easier when you can rely on the fact that the side effects from contract predicates are evaluated as any other expression. This is a more intuitive model, in the spirit of an imperative language, such as C++. It follows the existing practice with `assert()`: it allows side effects, but a lot of advice comes with it, saying that side effects in the predicate cannot be relied upon.

The arguments in favor of duplicating or eliminating side effects are:

1. Leaving room for implementations that might need to execute predicates twice in some cases.
2. Exerting a pressure on programmers, so that they provide side-effect-free contract predicates.

Some implementation strategies may need to evaluate the same predicate in a precondition twice. For direct function calls, an implementation can easily insert the instrumentation code in the caller. This is desired as it gives better diagnostics (the file name and line number of the caller who violated the precondition). However, this is impossible when a function is called indirectly, either through a pointer or `std::function`: from the pointer signature we do not know if a function called has a precondition or not. To address that case, one thing an implementation could do is to compile the pre- and post-condition checks into the function body. This would give the result that the pre-/post-conditions are checked normally when the function is called through an indirection, but are checked twice when the function is called directly: once in the caller, and once inside the function body. We may want to enable such implementation strategies. The consequence for the programmer is that when the predicate has side effects, these effects occur twice.

The implementation might want to provide a "mixed mode" where different translation units are compiled in different modes. For the MVP, we define this situation as IF-NDR, but in the future we may want to allow it. At this point we do not know the implementation difficulties that might arise from the mixed modes, the necessity to duplicate side effects in certain cases might be one of the results.

The mere possibility that the side effects of contract predicates can disappear in *Eval_and_abort* mode — provided that the programmer is aware of this — should be a strong incentive not put any "essential" side effects (e.g., logging is not such essential side effect) in their predicates. As we cannot easily prevent side effects statically in C++, this might be our best bet.

Note that runtime performance of the program in *Eval_and_abort* mode is *not* a motivation for side effect removal.

It should also be noted that the removal of side effects and the duplication of side effects are two decisions that can be made separately.

Also, it should be noted that the side effect elimination/duplication, if deemed useful, need to be added in the MVP and cannot be added later, because then it would be a breaking change for people who rely on these side effects.

3. Acknowledgements

{ack}

This paper is a summary of discussion between SG21 members about side effects in contract predicates. Gabriel Dos Ries suggested and explained the approach of making side effects ill-formed.

4. References

{ref}

- [N4302] — "Technical Specification for C++ Extensions for Transactional Memory"
(<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4302.pdf>).
- [P0542R5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++"
(<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html>).
- [P1728R0] — Andrzej Krzemiński, "Preconditions, axiom-level contracts and assumptions – an in depth study"
(<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1728r0.html>).
- [P2182R0] — Andrzej Krzemiński, Ryan McDougall, Joshua Berne, "Contract Support: Defining the Minimum Viable Feature Set"
(<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2182r0.html>).
- [P2388R4] — Andrzej Krzemiński, Gašper Ažman, "Minimum Contract Support: either *No_eval* or *Eval_and_abort*"
(<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2388r4.html>).
- [P2521R2] — Gašper Ažman, Joshua Berne, Bronek Kozicki, Andrzej Krzemiński, Ryan McDougall, Caleb Sunstrum, "Contract support — Working Paper"
(<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2521r2.html>).